

Deep Learning Based Model Predictive Control of Nonlinear Systems

October 4, 2026

1 Dataset Generation

1.1 Dynamics Equations

Projenin ilk aşaması mükemmel bir veri seti üretmektir. Bu veri setinin üzerinde deep learning modeli kuracağız. Kurduğumuz model inverted cart pendulum sisteminin diferensiyal denklemlerini, gerçekçi fizikini birebir yansıtabilmeli. Bunun için **dataset generation** adımı ilk ve aslında tüm projenin gidişatını belirleyen en önemli adımdır. Bunun için bu adıma çok özen göstermek lazım.

Kuracağımız modelin; sistem dinamiklerini nasıl öğrendiğini test etmek için farklı başlangıçlarda simulasyonlar yapmamız gerekecektir. Mesela, tam yukarı pozisyonda bırakarak sarkacın sadece yer çekimi kuvveti altında nasıl sallanarak, en sonda stable - sükunet haline döndüğünü test edip görmemiz gerekiyor. Bunun için sistem fiziki gerçekçi olmak zorunda. O yüzden de, **sistemimize friction - sürtünme kuvvetlerini: yani, arabanın rayla ve sarkaç eklemindeki sürtünmelerini (sürtünme katsayılarını) eklemek zorundayız.**

$$\text{Pendulum Sürtünmesi: } \tau_{eff} = \frac{M_t}{m_{pend}L} b_p \dot{\theta}$$

$$\text{Araba sürtünmesi: } F_{eff} = F - b_c \dot{x}$$

Bu formülleri kullanarak, diferensiyal denklemlerimizi aşağıdaki gibi yazabiliriz:

$$\ddot{\theta} = \frac{M_t g \sin \theta - \tau_{eff} - \cos \theta (F_{eff} + m_{pend} L \dot{\theta}^2 \sin \theta)}{L (M_t - m_{pend} \cos^2 \theta)}$$

$$\ddot{x} = \frac{F_{eff} + m_{pend} L (\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta)}{M_t}$$

Bu formüllerde, Total Mass $M_t = M_{cart} + m_{pend}$, Effective linear force (yani sürtünmeyi çıktığımızda etki eden force) F_{eff} , cart friction coefficient b_c , rotational damping (yani, dönen nesneye - pendulumuna dönüş yönünün tersinde etki eden ters kuvvet) τ_{eff} ve joint friction coefficient b_p olarak belirlenmiştir.

State vektörümüz $\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}]^T$; x - position, $\dot{x}$ - velocity, θ - angle ve $\dot{\theta}$ - angular velocity olarak belirlenmiştir.

Next state vektörü, state vektörünün türevine eşit oluyor ve continuous time-dan discrete time-a Runge Kutta 4 metodu ile geçiriliyor. RK4 metodu ile time intervali $\Delta t = 0.02s$ kabulederek $(\mathbf{s}, F) = [\dot{x}, \ddot{x}, \dot{\theta}, \ddot{\theta}]^T$ vektörünü, yani next state vektörünü aşağıdaki formüllerle alabiliriz:

$$k_1 = f(\mathbf{s}_t, F)$$

$$k_2 = f\left(\mathbf{s}_t + \frac{\Delta t}{2}k_1, F\right)$$

$$k_3 = f\left(\mathbf{s}_t + \frac{\Delta t}{2}k_2, F\right)$$

$$k_4 = f(\mathbf{s}_t + \Delta t k_3, F)$$

$$\text{Next State: } \mathbf{s}_{t+1} = \mathbf{s}_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Yani, önce k_1 , k_2 , k_3 ve k_4 sloplarını elde ediyoruz: 4 farklı noktadaki değeri bulup, weighted average alıyoruz. Bu bize Euler yönteminden çok daha güçlü, gerçeğe yakın sonuçlar verecektir.

1.2 Dataset Distribution

Datasetin dinamiklerini formüllerle belirlemek ilk aşamaydı. İkinci aşama ise farklı başlangıç noktaları vererek, o noktalarda sarkacın hareketlerini takip etmek. Bizim kabulettiğimiz sarkaç 360 derece dönebilen bir sarkaç modeli. Yalnız MPC controlleri ile kuracağımız control sayesinde biz daha çok hedeflediğimiz dikey pozisyon etrafındaki açılarla çalışacağız. Yani biz daha çok 0 derecenin yakın etraflarında sarkacın nasıl tepki verdiğini gözlemlemeliyiz ki, bu civarda modelimiz dinamikleri kusursuz şekilde öğrensin. Bunun için datasetin 75% oranını 0 angle civarında ele alacağız. Aşağıdaki Python kodu bu bölgeyi hedefliyor.

```
# 75% Operating Region
N_op = int(CHUNK_SIZE * 0.75)
x_op = np.random.uniform(-2.0, 2.0, size=(N_op, 1))
x_dot_op = np.random.normal(0.0, 1.0, size=(N_op, 1))
theta_op = np.random.normal(0.0, 0.2, size=(N_op, 1))
theta_dot_op = np.random.normal(0.0, 1.5, size=(N_op, 1))
forces_op = np.random.normal(0.0, 3.0, size=(N_op, 1))
```

Datasetin diğ er 25%-lik kısmında ise broad area - yani tüm space ele alınmıştır. Bu alanda artık pendulumun -180 ile 180 açıları arasındaki tüm noktalardaki hareketi gözlemlenecektir. Aşağıdaki Python kodu da bu alanı belirliyor:

```
# 25% Wide Exploration
N_wide = CHUNK_SIZE - N_op
x_wide = np.random.uniform(-5.0, 5.0, size=(N_wide, 1))
x_dot_wide = np.random.uniform(-10.0, 10.0, size=(N_wide, 1))
theta_wide = np.random.uniform(-np.pi, np.pi, size=(N_wide, 1))
theta_dot_wide = np.random.uniform(-20.0, 20.0, size=(N_wide, 1))
forces_wide = np.random.uniform(-15.0, 15.0, size=(N_wide, 1))
```